

ACARA PRAKTIKUM 6

JARINGAN SYARAF TIRUAN (JST)

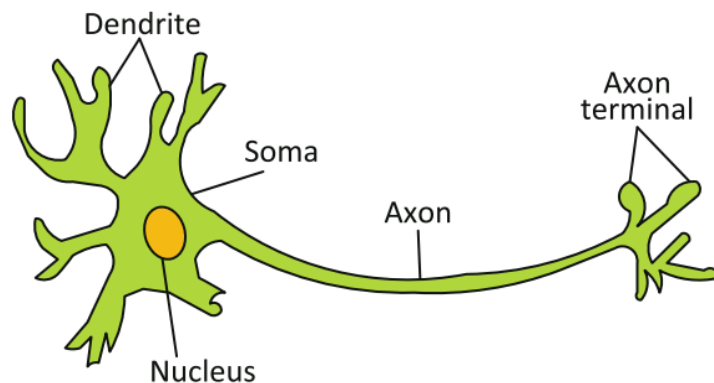
TUJUAN

1. Mengetahui konsep JST dan metode pembelajarannya
2. Menerapkan konsep JST dalam kode Python

DASAR TEORI

1. Pengenalan Jaringan Syaraf Tiruan (JST)

Jaringan Syaraf Tiruan (JST) adalah paradigma pemrosesan suatu informasi yang terinspirasi oleh sistem sel syaraf biologi (**neuron**), sama seperti otak dalam memproses suatu informasi. Setiap sel (neuron) memiliki satu **nukleus** (soma), bertugas memproses informasi, informasi diterima oleh **dendrit**, dan disebarkan melalui **akson**. Pertemuan informasi antar syaraf berada di **sinapsis**. Struktur sel syaraf (neuron) ditunjukkan dalam Gambar berikut,



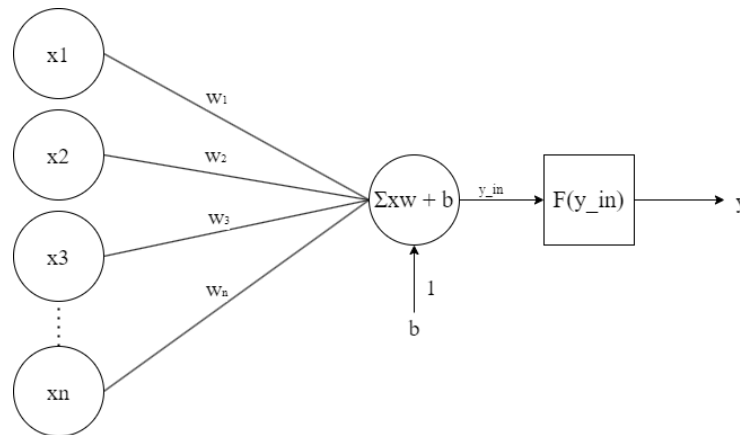
JST juga memproses informasi dalam struktur terkecil (**neuron**), hubungan antar neuron menyimpan **bobot** tertentu yang dibangkitkan secara acak. Informasi yang disalurkan dalam neuron kemudian dijumlahkan dan disandingkan dengan nilai ambang batas (**threshold**) melalui **fungsi aktivasi**. Jika informasi tersebut melewati threshold, maka neuron tersebut akan diaktifkan dan menyalurkan informasi ke neuron lain yang berhubungan. Bobot dan threshold bersifat dinamis dalam JST dan bisa diperbaiki melalui berbagai mekanisme pembelajaran. JST sangat baik untuk diaplikasikan dalam masalah klasifikasi, asosiasi, dan optimasi.

2. Fungsi Aktivasi

Fungsi aktivasi adalah fungsi yang digunakan dalam JST untuk mengaktifkan atau tidak mengaktifkan neuron. Sebuah neuron akan mengolah n input ($x_1, x_2, x_3, \dots, x_n$) yang masing-masing memiliki bobot ($w_1, w_2, w_3, \dots, w_n$) dan bobot bias b . Secara matematis dapat dituliskan sebagai berikut,

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

Kemudian, fungsi aktivasi F akan mengaktivasi y_{in} menjadi output jaringan y , seperti ditunjukkan pada gambar berikut,



Delta Rule digunakan untuk mengubah bobot yang menghubungkan antara jaringan input ke unit output dengan nilai target. Algoritma Delta Rule yang mengubah bobot ke- i (untuk setiap pola) adalah sebagai berikut,

$$\Delta w_i = \alpha (t_i - y_i) * x_i$$

Dengan,

Δw_i : perubahan bobot

α : learning rate

t_i : target

y_i : input jaringan ke unit output

Bobot baru didapat dari bobot lama ditambah perubahan bobot atau,

$$w_i = w_i + \Delta w_i$$

Adapun fungsi aktivasi yang digunakan dalam JST yaitu sebagai berikut,

- Fungsi undak biner

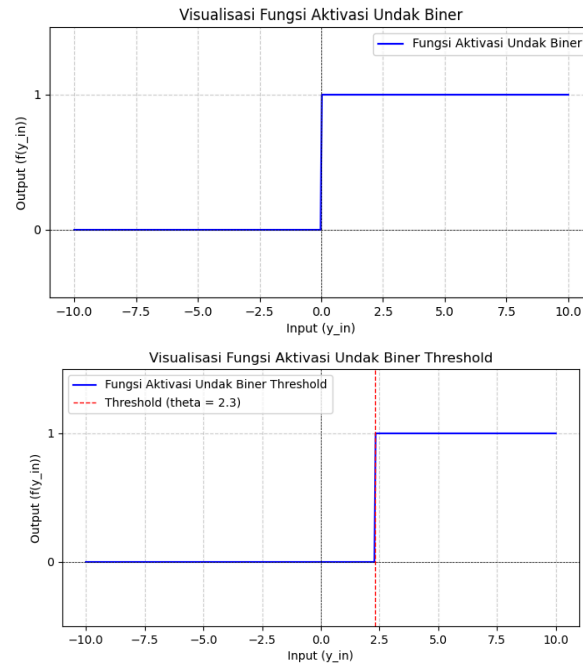
Fungsi undak biner (step function) sering digunakan pada jaringan dengan lapisan tunggal untuk mengkonversi input dari suatu variabel yang bernilai kontinu ke suatu output biner (0 atau 1), secara matematis seperti pada persamaan berikut,

$$y = \{0, \text{jika } y_{in} \leq 0; 1, \text{jika } y_{in} > 0$$

Fungsi undak biner juga bisa dimodifikasi menggunakan threshold θ , seperti pada persamaan berikut,

$$y = \{0, \text{jika } y_{in} < \theta; 1, \text{jika } y_{in} \geq \theta$$

Persamaan tersebut bisa divisualisasikan seperti gambar berikut,



- Fungsi bipolar

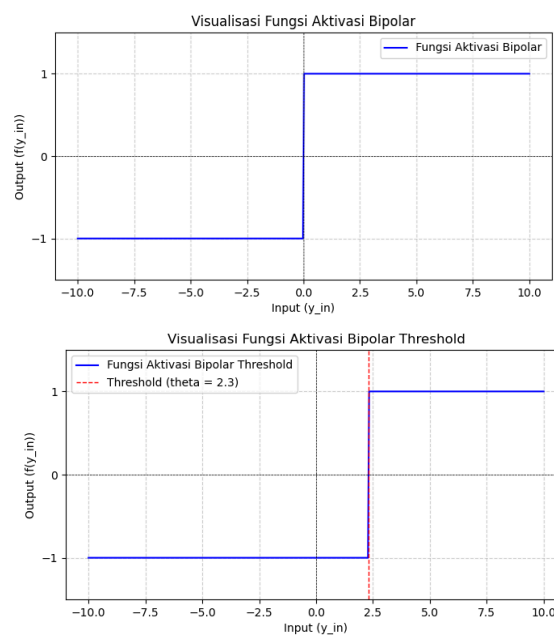
Fungsi bipolar serupa seperti fungsi undak biner hanya saja output yang dihasilkan berupa bipolar (-1, 0, 1), secara matematis seperti pada persamaan berikut,

$$y = \{-1, \text{ jika } y_{in} < 0; 0, \text{ jika } y_{in} = 0; 1, \text{ jika } y_{in} > 0$$

Fungsi bipolar juga bisa dimodifikasi menggunakan threshold θ , seperti pada persamaan berikut,

$$y = \{-1, \text{ jika } y_{in} < \theta; 0, \text{ jika } y_{in} = \theta; 1, \text{ jika } y_{in} > \theta$$

Persamaan tersebut bisa divisualisasikan seperti gambar berikut,

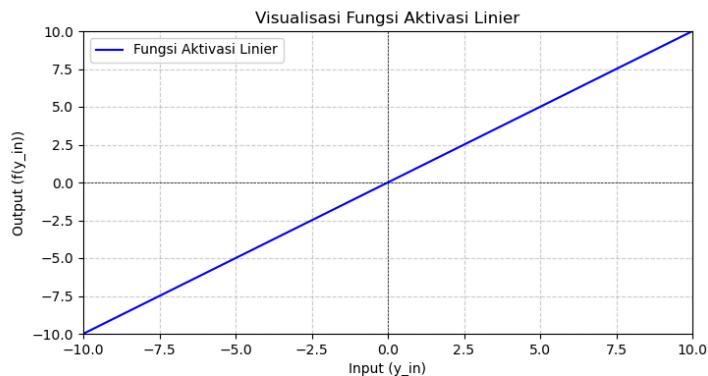


- Fungsi linier

Fungsi linier atau fungsi identitas akan menghasilkan nilai output yang sama dengan nilai input, secara matematis seperti pada persamaan berikut,

$$y = y_{in}$$

Persamaan tersebut bisa divisualisasikan seperti gambar berikut,

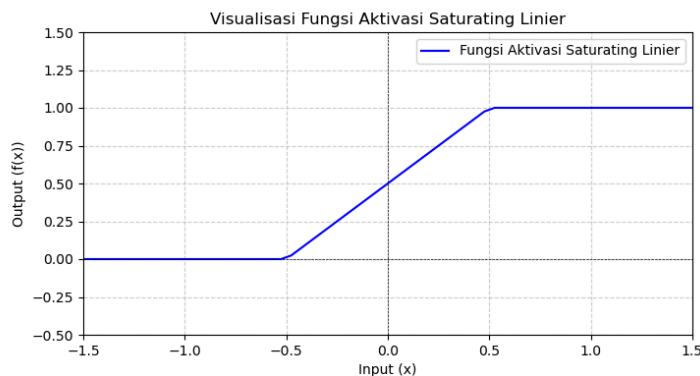


- Fungsi saturating linier

Fungsi ini akan menghasilkan nilai output 0 jika nilai input kurang dari -0.5 dan menghasilkan nilai output 1 jika nilai input lebih dari 0.5. Sementara itu, jika nilai input berada diantara -0.5 dan 0.5 maka nilai outputnya akan sama dengan nilai input ditambah 0.5. Secara matematis seperti pada persamaan berikut,

$$y = \{0, \text{ jika } y_{in} \leq -0.5; y_{in} + 0.5, \text{ jika } -0.5 \leq y_{in} \leq 0.5; 1, \text{ jika } y_{in} \geq 0.5$$

Persamaan tersebut bisa divisualisasikan seperti gambar berikut,

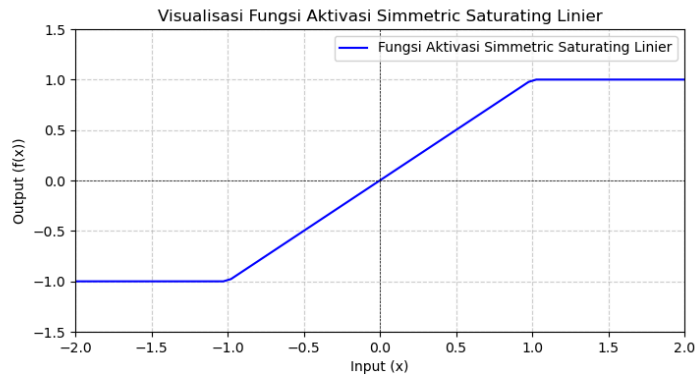


- Fungsi simmetric saturating linier

Fungsi ini akan menghasilkan nilai output -1 jika nilai input kurang dari -1 dan menghasilkan nilai output 1 jika nilai input lebih dari 1. Sementara itu, jika nilai input berada diantara -1 dan 1 maka nilai outputnya akan sama dengan nilai input. Secara matematis seperti pada persamaan berikut,

$$y = \{-1, \text{ jika } y_{in} \leq -1; y_{in}, \text{ jika } -1 \leq y_{in} \leq 1; 1, \text{ jika } y_{in} \geq 1$$

Persamaan tersebut bisa divisualisasikan seperti gambar berikut,

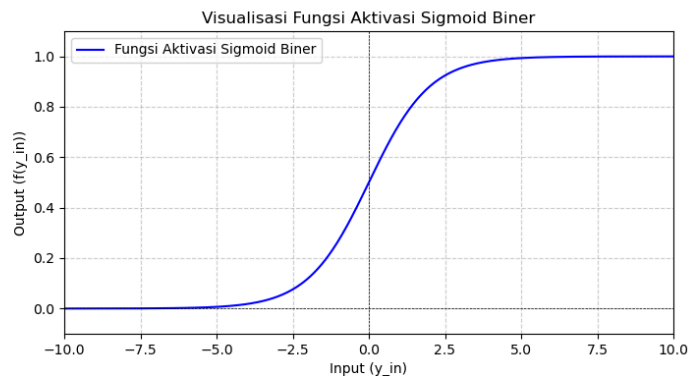


- Fungsi sigmoid biner

Fungsi ini akan menghasilkan nilai output yang berada diantara 0 sampai 1 untuk membatasi nilai output dan memudahkan interpretasi secara probabilitas, secara matematis seperti pada persamaan berikut,

$$y = \frac{1}{1+e^{-y_{in}}}$$

Persamaan tersebut bisa divisualisasikan seperti gambar berikut,

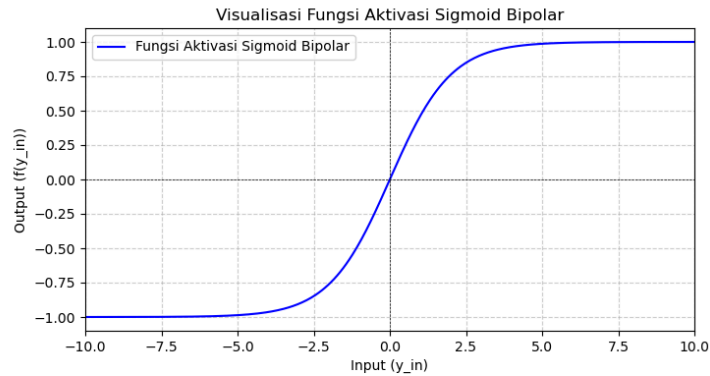


- Fungsi sigmoid bipolar

Fungsi sigmoid bipolar dikenal juga dengan nama fungsi tanh. Fungsi ini akan menghasilkan nilai output yang berada diantara -1 sampai 1 untuk membatasi nilai output dan memudahkan interpretasi secara probabilitas, secara matematis seperti pada persamaan berikut,

$$y = \frac{1-e^{-y_{in}}}{1+e^{-y_{in}}}$$

Persamaan tersebut bisa divisualisasikan seperti gambar berikut,



3. Masalah OR dan Pengenalan Perceptron

Perceptron adalah sebuah sistem yang belajar menggunakan contoh berlabel (pembelajaran terawasi) dari vektor fitur dengan memetakan input-input ke dalam label kelas output yang sesuai. Dalam bentuk paling sederhananya, Perceptron memiliki N node input yang dimuat dalam jaringan berlayer tunggal dan setiap input diberikan bobot yang sesuai. Setiap pasangan input dan bobot dipetakan menjadi satu node output dengan menjumlahkan dan memberlakukan fungsi aktivasi. Perceptron menghasilkan output berupa linier seperti $[0,1]$ untuk biner dan $[-1,1]$ untuk bipolar.

Pada masalah OR berikut, diberikan data berupa input dan target bipolar, learning rate 0.01, bobot awal 0, dan epoch 10.

x_1	x_2	t
1	1	1
1	-1	1
-1	1	1
-1	-1	-1

- 1) Buat file Perceptron.py, kemudian definisikan kelas Perceptron dan konstruktor nilai-nilai awal.

```
# Import library
import numpy as np
import matplotlib.pyplot as plt

# Buat kelas Perceptron
class Perceptron:
    # Simpan learning rate dan max epoch dalam konstruktor
    def __init__(self, alpha=0.1, epoch=10):
        self.alpha = alpha
        self.epoch = epoch
```

- 2) Pada kelas Perceptron buat fungsi untuk menghitung y_{in} dan fungsi aktivasi.

```
# Fungsi menghitung nilai  $y_{in}$  atau net
def weighted_sum(self, X):
    return np.dot(X, self.w[1:]) + self.w[0]

# Fungsi menerapkan fungsi aktivasi bipolar
def predict(self, X):
    return np.where(self.weighted_sum(X) >= 0.0, 1, -1)
```

- 3) Pada kelas Perceptron buat juga fungsi untuk simulasi garis pemisah model Perceptron.

```
# Fungsi membuat simulasi garis pemisah data
def plot_decision_boundary(self, X, t, epoch):
    # Membuat titik data input
    plt.scatter(X[:, 0], X[:, 1], c=t.ravel(), marker='o',
edgecolors='k', cmap=plt.cm.RdYlBu)

    # Menentukan limit tampilan bidang grafik
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    # Membuat garis pemisah
    x_vals = np.linspace(x_min, x_max, 100)
    y_vals = -(self.w_[0] + self.w_[1] * x_vals) / self.w_[2]
    plt.plot(x_vals, y_vals, 'b-', label=f'Decision boundary (Epoch
{epoch+1})')

    plt.xlim(x_min, x_max)
    plt.ylim(y_min, y_max)
    plt.title(f"Decision Boundary Pada Epoch {epoch+1}")
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.legend()
    plt.show()
```

- 4) Pada kelas Perceptron buat fungsi utama Perceptron.

- Inisialisasi bobot dan bias = 0.
- Operasikan $y_{in} = b + \sum_{i=1}^n x_i w_i$ untuk setiap input dengan bobot dan bias awal.
- Aktivasi hasil y_{in} dengan fungsi aktivasi bipolar.
- Jika error = 0 maka bobot dan bias tidak berubah, tetapi jika error $\neq 0$ maka dilakukan perubahan bobot dan bias.
- Lakukan perubahan bobot $w_i = w_i + \Delta w_i$ dengan $\Delta w_i = \alpha(t_i - y_i) * x_i$ dan bias $b_i = b_i + \Delta b_i$ dengan $\Delta b_i = \alpha(t_i - y_i)$.
- Cek kondisi berhenti, jika max epoch tercapai atau Sum Square Error(SSE) = 0.

```
# Fungsi utama Perceptron
def fit(self, X, t):
    # Inisialisasi bobot dan bias awal = 0
    self.w_ = np.zeros(1 + X.shape[1])

    # Menyimpan hasil pada HasilPerceptron.txt
    with open("HasilPerceptron.txt", "w") as f:
        f.write("Masalah OR dengan Perceptron\n")
        f.write("-----\n")
        f.write(f"Input : \n{X}\n")
        f.write(f"Target: \n{t}\n")
        f.write(f"Bobot awal : {self.w_[1:]} \n")
        f.write(f"Bias awal : {self.w_[0]} \n")
        f.write(f"Learning rate : {self.alpha} \n")
        f.write(f"Max Epoch : {self.epoch} \n")
```

```

# Iterasi Perceptron(Epoch)
for epoch in range(self.epoch):
    f.write(f"\nEpoch {epoch + 1}/{self.epoch}\n")
    f.write("-----\n")
    error = np.array([])

    # Iterasi setiap pasang matriks input dengan targetnya
    for xi, target in zip(X, t):
        # Periksa input dengan model Perceptron
        y_pred = self.predict(xi)

        # Periksa apakah output sesuai target, jika iya maka
        error = 0
        error = np.append(error, target - y_pred)

        # Modifikasi bobot dengan Delta Rule, jika error = 0 maka
        bobot tetap
        update = self.alpha * error[-1]

        # Modifikasi bobot w
        self.w_[1:] += update * xi

        # Modifikasi bias b
        self.w_[0] += update

        # Menuliskan hasil tiap iterasi input pada satu epoch
        f.write(f"Input: {xi}, Target: {target}, Predict:
{y_pred}, Error: {error[-1]}, Bobot: {self.w_[1:]}, Bias:
{self.w_[0]}\n")

        # Simulasikan garis pemisah model Perceptron
        self.plot_decision_boundary(X, t, epoch)

        # Menuliskan penjumlahan kuadrat error setiap input
        f.write(f"Sum Square Error(SSE): {sum(error ** 2)}\n")

        # Periksa kondisi berhenti, jika penjumlahan kuadrat error
        setiap input = 0 atau max epoch tercapai
        if sum(error ** 2) == 0 or epoch + 1 == self.epoch:

f.write("-----\n")
f.write(f"Pelatihan berhenti pada epoch ke-{epoch + 1}
karena ")
        f.write("Sum Square Error(SSE) mencapai target.\n" if
epoch + 1 != self.epoch else "max epoch tercapai.\n")
        # Menuliskan bobot terakhir
        f.write(f"\nBobot akhir      :{self.w_[1:]}\n")
        f.write(f"Bias akhir       :{self.w_[0]}\n")
        break

```

- 5) Buat file baru Perceptron_or.py dan import model Perceptron sebelumnya, kemudian inisialisasi input, target, learning rate, dan epoch sesuai masalah yang diberikan.

```

# Import library
import numpy as np
import Perceptron as p

```



```
# Inisialisasi input dan target
X = np.array([[1,1], [1,-1], [-1,1], [-1,-1]])
t = np.array([[1], [1], [1], [-1]])

# Pemanggilan model Perceptron
model = p.Perceptron(alpha=0.1, epoch=10)
model.fit(X, t)
```

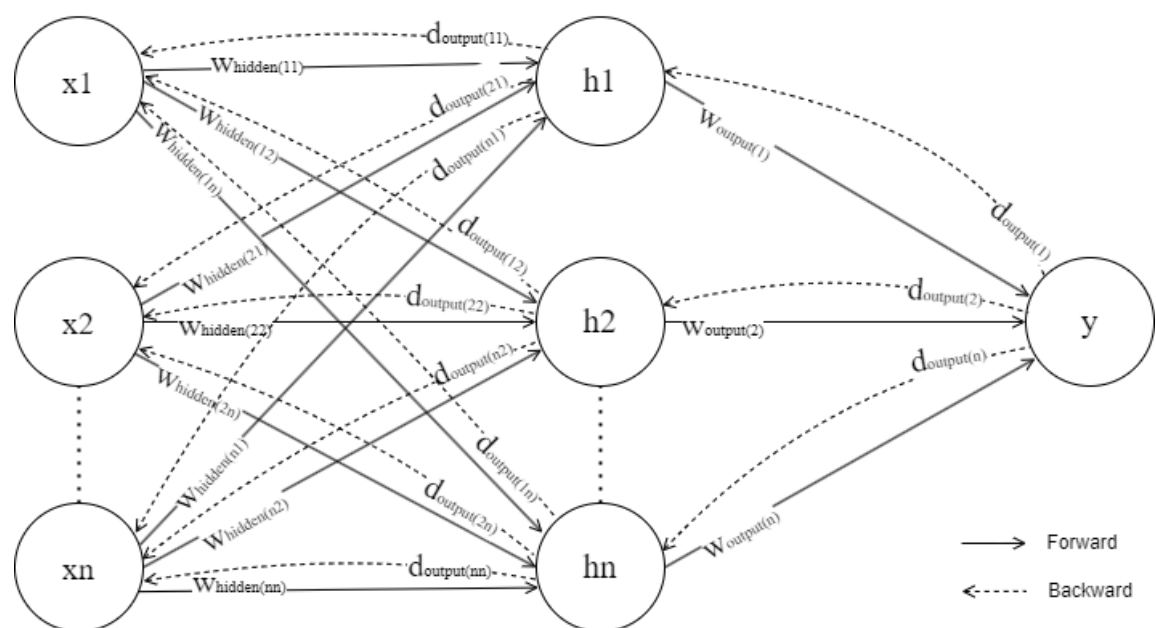
- 6) Jalankan program pada file Perceptron_or.py, simulasi grafik setiap epoch akan ditampilkan dan hasil perhitungan dapat dilihat pada file HasilPerceptron.txt.
- 7) Contoh perhitungan secara manual seperti tabel berikut.

X1	X2	t	y_in	y	$\Delta w1$	$\Delta w2$	Δb	w1	w2	b
Inisialisasi (Epoch 1)										
1	1	1	0	1	0	0	0	0	0	0
1	-1	1	0	1	0	0	0	0	0	0
-1	1	1	0	1	0	0	0	0	0	0
-1	-1	-1	0	1	0.2	0.2	-0.2	0.2	0.2	-0.2
Inisialisasi (Epoch 2)										
1	1	1	0.2	1	0	0	0	0.2	0.2	-0.2
1	-1	1	-0.2	-1	0.2	-0.2	0.2	0.4	0	0
-1	1	1	-0.4	-1	-0.2	0.2	0.2	0.2	0.2	0.2
-1	-1	-1	-0.2	-1	0	0	0	0.2	0.2	0.2
Inisialisasi (Epoch 3)										
1	1	1	0.6	1	0	0	0	0.2	0.2	0.2
1	-1	1	0.2	1	0	0	0	0.2	0.2	0.2
-1	1	1	0.2	1	0	0	0	0.2	0.2	0.2
-1	-1	-1	-0.2	-1	0	0	0	0.2	0.2	0.2

4. Masalah XOR dan Pengenalan Backpropagation

Algoritma Backpropagation bekerja seperti Perceptron, termasuk model pembelajaran terawasi dan membutuhkan label pada data pelatihannya. Hal yang membedakan keduanya, pada Backpropagation terdapat layer tambahan di antara layer input dan layer output, layer tambahan tersebut biasa disebut dengan hidden layer dan menjadi dasar model jaringan multi layer. Selain itu pada mekanisme perubahan bobotnya, terdapat perambatan mundur setelah dihasilkan output untuk memberikan respon dalam perubahan bobot. Fungsi aktivasi yang digunakan pada algoritma ini yaitu fungsi sigmoid dan tanh.

Arsitektur model Backpropagation yaitu sebagai berikut,



Pada masalah XOR, model single layer seperti Perceptron tidak bisa mengenali permasalahan karena keterbatasan dalam pemisahan data yang menggunakan garis lurus/linier, hidden layer dibutuhkan untuk menyelesaikan permasalahan tersebut.

Diketahui permasalahan XOR, dengan ketentuan input dan target bipolar, learning rate 0.3, max epoch 1000, dan target error 0.001.

x_1	x_2	t
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1

- 1) Buat file Backpropagation.py kemudian definisikan kelas Backpropagation dan inisialisasi awal pada konstruktor. Bobot dan bias dibuat secara random sesuai jumlah neuron tiap layer.

```
# Import library
import numpy as np
import matplotlib.pyplot as plt

# Buat kelas Backpropagation
class Backpropagation:
    # Simpan learning rate, epoch, dan target error dalam konstruktor
    # serta inisialisasi bobot dan bias awal random
    def __init__(self, alpha, epoch, target_error):
        self.alpha = alpha
        self.epoch = epoch
        self.target_error = target_error

        self.n_input = 2
        self.n_hidden = 2
        self.n_output = 1

        self.w_hidden = np.random.rand(self.n_input, self.n_hidden)
        self.b_hidden = np.random.rand(1, self.n_hidden)

        self.w_output = np.random.rand(self.n_hidden, self.n_output)
        self.b_output = np.random.rand(1, self.n_output)
```

- 2) Pada kelas Backpropagation, buat fungsi untuk melakukan fungsi aktivasi dan turunan dari fungsi tersebut. Untuk input dan target bersifat bipolar, gunakan fungsi sigmoid bipolar atau tanh. Sementara itu, untuk input dan target biner menggunakan fungsi sigmoid biner.

```
# Fungsi menerapkan fungsi aktivasi sigmoid bipolar atau tanh
def bi_sigmoid(self, x):
    return np.tanh(x)

# Fungsi menerapkan turunan fungsi aktivasi sigmoid bipolar atau tanh
# (asumsi x = output sigmoid bipolar/tanh)
def deriv_bi_sigmoid(self, x):
    return 1 - x**2
```

- 3) Pada kelas Backpropagation, buat juga fungsi yang melakukan simulasi penurunan Sum Square Error(SSE) setiap epoch.

```
# Fungsi membuat simulasi perbaikan bobot dan bias
```

```

def plot_error(self, x, epoch):
    plt.plot(range(1, epoch + 1), x, linestyle='-', color='b',
             label='Error')

    final_error = x[-1]
    plt.annotate(f'Epoch {epoch}, Error: {final_error:.4f}',
                xy=(epoch, final_error),
                xytext=(epoch - len(x) * 0.2, final_error + 0.05),
                arrowprops=dict(facecolor='black', arrowstyle="->"),
                fontsize=10, color='red')

    plt.title('Perbaikan Error Setiap Epoch')
    plt.xlabel('Epoch')
    plt.ylabel('Sum Square Error(SSE)')
    plt.grid(True)
    plt.legend()
    plt.show()

```

4) Pada kelas Backpropagation, buat fungsi utama Backpropagation.

- Inisialisasi bobot dan bias secara random dari 0-1.
- Terdapat operasi forward dan backward propagation.
 - Pada operasi forward propagation, operasikan

$$h_{in} = b_{(hidden)} + \sum_{i=1}^n x_i w_{(hidden)i} \text{ input layer dengan hidden layer.}$$

- Aktivasi hidden layer dengan fungsi tanh karena data yang bersifat bipolar, jika datanya biner menggunakan fungsi sigmoid. $h = \tanh(h_{in})$.

- Selanjutnya pada forward propagation operasikan

$$y_{in} = b_{(output)} + \sum_{i=1}^n h_i w_{(output)i} \text{ hidden layer dengan output layer.}$$

- Aktivasi output layer dengan fungsi tanh. $y = \tanh(y_{in})$.
- Pada operasi backward propagation, hitung error dari output layer.

$$error = target_i - y_i.$$

- Hitung delta output layer berdasarkan errornya.

$$\delta_{(output)} = error * \tanh^{-1}(y_{in}) \text{ atau } \delta_{(output)} = error * (1 - y^2).$$

- Hitung error dari hidden layer. $error_{(hidden)} = \sum_{i=1}^n \delta_{(output)i} w_{(output)i} T.$

- Hitung delta hidden layer berdasarkan errornya.

$$\delta_{(hidden)} = error_{(hidden)} * \tanh^{-1}(h_{in}) \text{ atau}$$

$$\delta_{(output)} = error * (1 - h^2).$$

- Lakukan perubahan bobot output layer

$$w_{(output)i} = w_{(output)i} + \Delta w_{(output)i} \text{ dengan}$$

$$\Delta w_{(output)i} = \sum_{i=1}^n (h_i \cdot T * \delta_{(output)i}) \alpha \text{ dan bias output layer}$$

$$b_{(output)i} = b_{(output)i} + \Delta b_{(output)i} \text{ dengan } \Delta b_{(output)i} = \sum_{i=1}^n (\delta_{(output)i}) \alpha$$

- Lakukan perubahan bobot hidden layer

$$w_{(hidden)i} = w_{(hidden)i} + \Delta w_{(hidden)i} \text{ dengan}$$

$$\Delta w_{(hidden)i} = \sum_{i=1}^n (x_i * \delta_{(hidden)i}) \alpha \text{ dan bias hidden layer}$$

$$b_{(hidden)i} = b_{(hidden)i} + \Delta b_{(hidden)i} \text{ dengan } \Delta b_{(hidden)i} = \sum_{i=1}^n (\delta_{(hidden)i}) \alpha$$

- Cek kondisi berhenti, jika Sum Square Error(SSE) lebih kecil dari target error atau max epoch tercapai.

```
# Fungsi utama Backpropagation
def fit(self, X, t):
    errors_per_epoch = []
    # Menyimpan hasil pada hasilBackpropagation.txt
    with open("hasilBackpropagation.txt", "w") as f:
        f.write("Masalah XOR dengan Backpropagation\n")
        f.write("-----\n")
        f.write(f"Input : \n{X}\n")
        f.write(f"Target : \n{t}\n\n")
        f.write(f"Bobot awal hidden layer : \n{self.w_hidden}\n\n")
        f.write(f"Bias awal hidden layer : \n{self.b_hidden}\n\n")
        f.write(f"Bobot awal output layer : \n{self.w_output}\n\n")
        f.write(f"Bias awal output layer : \n{self.b_output}\n\n")
        f.write(f"Learning rate : {self.alpha}\n")
        f.write(f"Max Epoch : {self.epoch}\n\n")
        # Iterasi Backpropagation(epoch)
        for epoch in range(self.epoch):
            f.write("-----\n")
            f.write(f"Epoch {epoch + 1}/{self.epoch}\n")
            f.write("-----\n")
            total_error = 0
            count = 1
            output = np.array([])
            # Iterasi setiap pasang matriks input dengan targetnya
            for xi, target in zip(X, t):
                f.write(f>Data ke-{count}\n")
                f.write("-----\n")
                f.write("----- Forward Propagation\n")
                f.write("-----\n")
                # Opeasikan input dengan hidden layer
                h_in = np.dot(xi, self.w_hidden) + self.b_hidden
                f.write(f"Operasi input ke hidden layer: \n{h_in}\n\n")
                # Aktivasi hasil operasi hidden layer dengan fungsi tanh
                h = self.bi_sigmoid(h_in)
                f.write(f"Aktivasi hidden layer: \n{h}\n\n")
                # Operasikan hidden layer dengan output layer
                y_in = np.dot(h, self.w_output) + self.b_output
                f.write(f"Operasi hidden ke output layer: \n{y_in}\n\n")
                # Aktivasi hasil operasi output layer dengan fungsi tanh
                y = self.bi_sigmoid(y_in)
                output = np.append(output, y)
                f.write(f"Aktivasi output layer: \n{y}\n")
                f.write("----- Backward Propagation\n")
                f.write("-----\n")
                # Hitung error output layer terhadap target
                error = target - y
```

```

total_error += np.sum(error**2)
f.write(f"Error:\n{error}\n\n")
# Operasikan error dengan turunan dari aktivasi output
layer
    d_y = error * self.deriv_bi_sigmoid(y)
    f.write(f"Delta output (d_y):\n{d_y}\n\n")

# Hitung error hidden layer
error_h = np.dot(d_y, self.w_output.T)
f.write(f"Error hidden layer (error_h):\n{error_h}\n\n")
# Operasikan error dengan turunan dari aktivasi hidden
layer
    d_h = error_h * self.deriv_bi_sigmoid(h)
    f.write(f"Delta hidden layer (d_h):\n{d_h}\n\n")

# Perbaiki bobot dan bias output layer
self.w_output += np.dot(h.T, d_y) * self.alpha
f.write(f"Bobot output layer baru
(w_output):\n{self.w_output}\n\n")
self.b_output += np.sum(d_y, axis=0, keepdims=True) *
self.alpha
f.write(f"Bias output layer baru
(b_output):\n{self.b_output}\n\n")
# Perbaiki bobot dan bias hidden layer
self.w_hidden += np.dot(xi.reshape(2,1), d_h) *
self.alpha
f.write(f"Bobot hidden layer baru
(w_hidden):\n{self.w_hidden}\n\n")
self.b_hidden += np.sum(d_h, axis=0, keepdims=True) *
self.alpha
f.write(f"Bias hidden layer baru
(b_hidden):\n{self.b_hidden}\n\n")

f.write("-----\n")
    count += 1
    # Hitung Sum Square Error(SSE) pada setiap epoch
    average_error = total_error / len(X)
    errors_per_epoch.append(average_error)
    f.write(f"Output      : {output.reshape(1,4)}\n")
    f.write(f"Sum Square Error(SSE) epoch ke-{epoch + 1}:
{average_error}\n")
    # Cek kondisi berhenti, jika Sum Square Error(SSE) kurang
dari target error atau max epoch tercapai
    if average_error < self.target_error or epoch + 1 ==
self.epoch:

f.write("-----\n")
-----\n")
        f.write(f"Pelatihan berhenti pada epoch ke-{epoch + 1}
karena ")
        f.write("Sum Square Error(SSE) mencapai target.\n" if
epoch + 1 != self.epoch else "max epoch tercapai.\n")
        f.write(f"Bobot akhir hidden layer
:\n{self.w_hidden}\n\n")
        f.write(f"Bias akhir hidden layer
:\n{self.b_hidden}\n\n")
        f.write(f"Bobot akhir output layer
:\n{self.w_output}\n\n")
        f.write(f"Bias akhir output layer      :\n{self.b_output}")

```

```
self.plot_error(errors_per_epoch, epoch + 1)
break
```

- 5) Buat file baru Backpropagation_xor.py dan import model Backpropagation sebelumnya, kemudian inisialisasi input, target, learning rate, epoch, dan target error sesuai masalah yang diberikan.

```
# Import library
import numpy as np
import Backpropagation as b

# Inisialisasi input dan target
X = np.array([[1, 1], [1, -1], [-1, 1], [-1, -1]])
t = np.array([[1], [1], [1], [-1]])

# Pemanggilan model Backpropagation
model = b.Backpropagation(alpha=0.3, epoch=1000, target_error=0.001)
model.fit(X, t)
```

- 6) Jalankan program pada file Backpropagation_xor.py, simulasi grafik setiap epoch akan ditampilkan dan hasil perhitungan dapat dilihat pada file HasilBackpropagation.txt.

TUGAS

Upload source code hasil pengerjaan studi kasus di atas ke GitHub dengan nama repositori sebagai berikut,

NIM-PraktikumKB-Pertemuan6

(Tugas diselesaikan di waktu praktikum sekarang juga)